

## PROGRAM-2

AIM: Write a program to compute CRC code for the polynomials CRC-12, CRC-16 and CRC CCIP

### DESCRIPTION:

Cyclic Redundancy Check is a method of detecting accidental changes/errors in the communication channel.

CRC uses **Generator Polynomial** which is available on both sender and receiver side. An example generator polynomial is of the form like  $x^3 + x + 1$ . This generator polynomial represents key 1011. Another example is  $x^2 + 1$  that represents key 101.

n : Number of bits in data to be sent from sender side.

k : Number of bits in the key obtained from generator polynomial.

#### **Sender Side (Generation of Encoded Data from Data and Generator Polynomial (or Key)):**

1. The binary data is first augmented by adding k-1 zeros in the end of the data
2. Use **modulo-2 binary division** to divide binary data by the key and store remainder of division.
3. Append the remainder at the end of the data to form the encoded data and send the same

#### **Receiver Side (Check if there are errors introduced in transmission)**

Perform modulo-2 division again and if the remainder is 0, then there are no errors.

### SOURCE CODE:

```
#include <stdio.h>
#include <string.h>
#define MAX_SIZE 100
// Function to compute CRC code
void computeCRC(char data2[], char polynomial2[], int dl, int pl)
{
    // Compute CRC
    for (int i = 0; i < dl; i++)
    {
        if (data2[i] == '1')
        {
            for (int j = 0; j < pl; j++)
            {
                data2[i + j] = (data2[i + j] == polynomial2[j]) ? '0' : '1';
            }
        }
    }
}

// Output CRC code
printf("%s", &data2[dl]);
}
```

```

int main() {
    char data[MAX_SIZE];
    char polynomial[] = "1101";
    char codedata[MAX_SIZE];
    // Input data
    printf("Enter data: ");
    scanf("%s", data);
    printf("Enter Polynomial: ");
    scanf("%s", polynomial);
    int dl = strlen(data);
    int pl = strlen(polynomial);
    for (int i = 0; i < pl - 1; i++)
    {
        data[dl + i] = '0';
    }
    printf("Data with zeros: %s\n\n", data);
    printf("CRC at sender side: ");
    computeCRC(data, polynomial, dl, pl);
    printf("\n\nEnter codeword (Data + CRC): ");
    scanf("%s", codedata);
    printf("CRC at receiver side: ");
    computeCRC(codedata, polynomial, dl, pl);
}

```

### **OUTPUT:**

Enter data: 111101

Enter Polynomial: 1101

Data with zeros: 111101000

CRC at sender side: 011

Enter codeword (Data + CRC): 111101011

CRC at receiver side: 000

## VIVA – QUESTIONS:

Explain following Error control methods

- Hamming code
- CRC
- Checksum

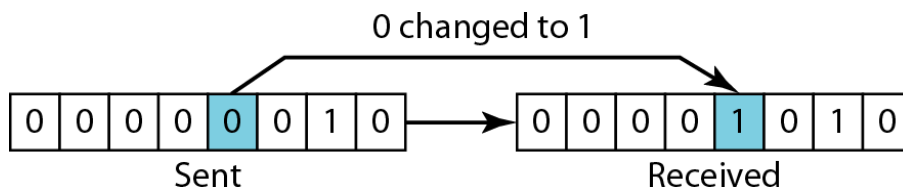
**Ans:** Networks must be able to transfer data from one device to another with acceptable accuracy. For most applications, a system must guarantee that the data received are identical to the data transmitted.

- Any time data are transmitted from one node to the next, they can become corrupted in passage. Many factors can alter one or more bits of a message. Some applications require a mechanism for detecting and correcting errors.

### Types of Errors:

#### Single-Bit Error:

In a single-bit error, only 1 bit in the data unit has changed.



(ASCII STX)

(ASCII LF)

#### Burst Error:

A burst error means that 2 or more bits in the data unit have changed.

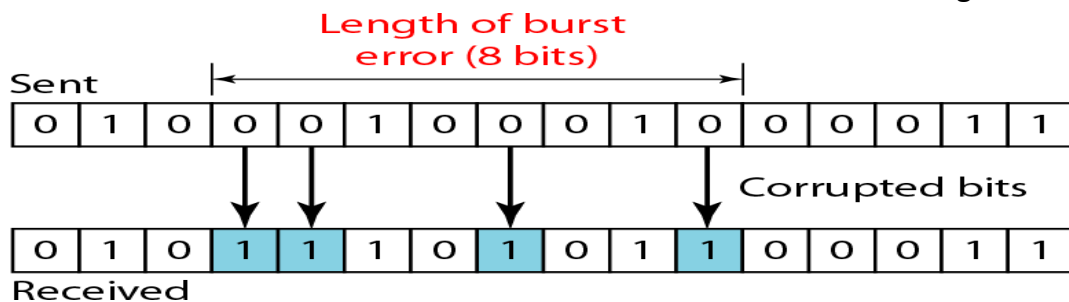
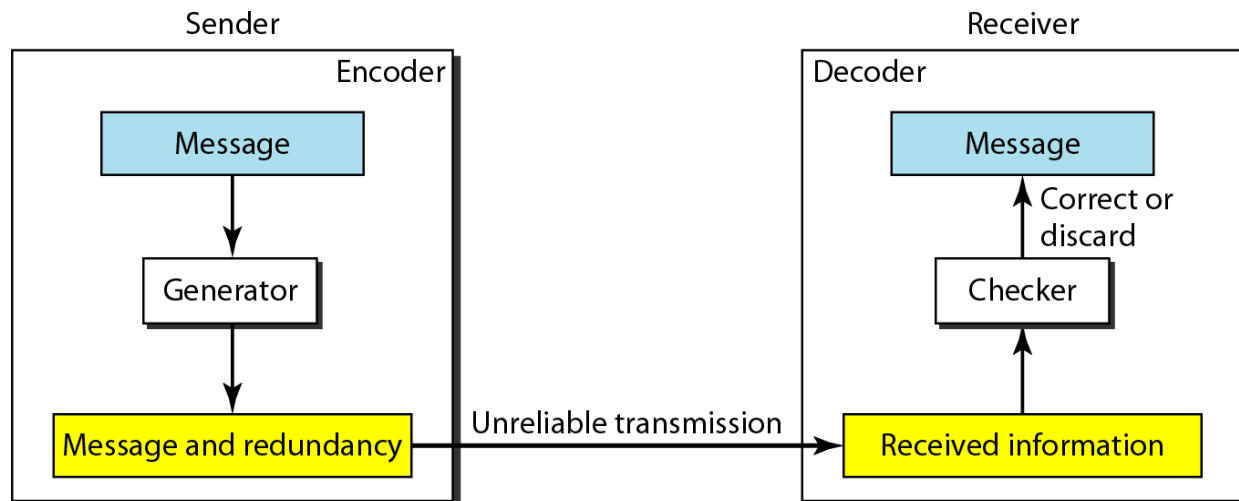


Fig: *Burst error of length 8*

## Redundancy:

To detect or correct errors, we need to send extra (redundant) bits with data.

### *The structure of encoder and decoder*



## Modular Arithmetic:

- In modular arithmetic, we use only a limited range of integers. We define an upper limit, called a modulus  $N$ . We then use only the integers 0 to  $N - 1$ , inclusive.
- In modulo- $N$  arithmetic, we use only the integers in the range 0 to  $N - 1$ , inclusive.
- Addition and subtraction in modulo arithmetic are simple. **Modulo-2 Arithmetic:**

Of particular interest is modulo-2 arithmetic. In this arithmetic, the modulus  $N$  is 2. We can use only 0 and 1. Operations in this arithmetic are very simple. The following shows how we can add or subtract 2 bits.

Adding: Subtracting:

$$0+0=0 \quad 0-0=0$$

$$0+1=1 \quad 0-1=1$$

$$1+0=1 \quad 1-0=1$$

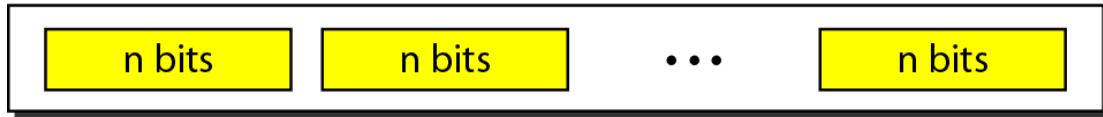
$$1+1=0 \quad 1-1=0$$

## BLOCK CODING:

In block coding, we divide our message into blocks, each of  $k$  bits, called data words. We add  $r$  redundant bits to each block to make the length  $n = k + r$ . The resulting  $n$ -bit blocks are called code words.



$2^k$  Datawords, each of  $k$  bits



$2^n$  Codewords, each of  $n$  bits (only  $2^k$  of them are valid)

**Data words and code words in block coding Hamming code:**

The Hamming distance between two words is the number of differences between corresponding bits.

*Let us find the Hamming distance between two pairs of words.*

1. The Hamming distance  $d$  (000, 011) is 2 because

$$000 \oplus 011 \text{ is } 011 \text{ (two 1s)}$$

2. The Hamming distance  $d$  (10101, 11110) is 3 because

$$10101 \oplus 11110 \text{ is } 01011 \text{ (three 1s)}$$

- The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.
- To guarantee the detection of up to  $s$  errors in all cases, the minimum Hamming distance in a block code must be  $d_{\min} = s + 1$ .
- To guarantee correction of up to  $t$  errors in all cases, the minimum Hamming distance in a block code must be  $d_{\min} = 2t + 1$ .

**CRC:**

Cyclic codes are special linear block codes with one extra property. In a cyclic code, if a code word is cyclically shifted (rotated), the result is another code word.

For example, if 1011000 is a code word and we cyclically left-shift, then 0110001 is also a code word.

In this case, if we call the bits in the first word  $a_0$  to  $a_6$  and the bits in the second word  $b_0$  to  $b_6$ , we can shift the bits by using the following:

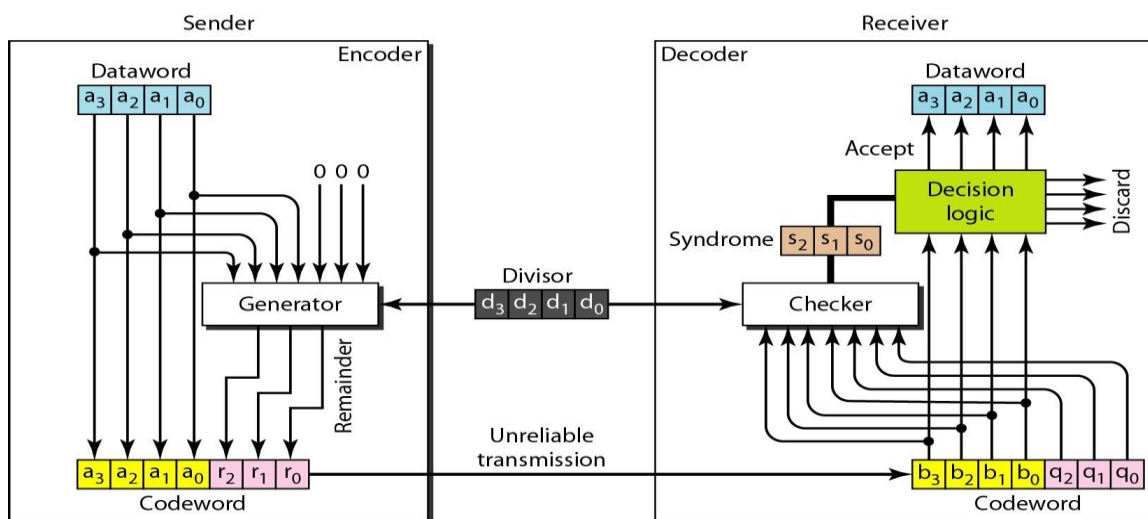
$$b_1 = a_0 \quad b_2 = a_1 \quad b_3 = a_2 \quad b_4 = a_3 \quad b_5 = a_4 \quad b_6 = a_5 \quad b_0 = a_6$$

In the rightmost equation, the last bit of the first word is wrapped around and becomes the first bit of the second word.

### A CRC code with C (7, 4)

| Dataword | Codeword | Dataword | Codeword |
|----------|----------|----------|----------|
| 0000     | 0000000  | 1000     | 1000101  |
| 0001     | 0001011  | 1001     | 1001110  |
| 0010     | 0010110  | 1010     | 1010011  |
| 0011     | 0011101  | 1011     | 1011000  |
| 0100     | 0100111  | 1100     | 1100010  |
| 0101     | 0101100  | 1101     | 1101001  |
| 0110     | 0110001  | 1110     | 1110100  |
| 0111     | 0111010  | 1111     | 1111111  |

### CRC encoder and decoder:



In the encoder, the data word has  $k$  bits (4 here); the code word has  $n$  bits (7 here).

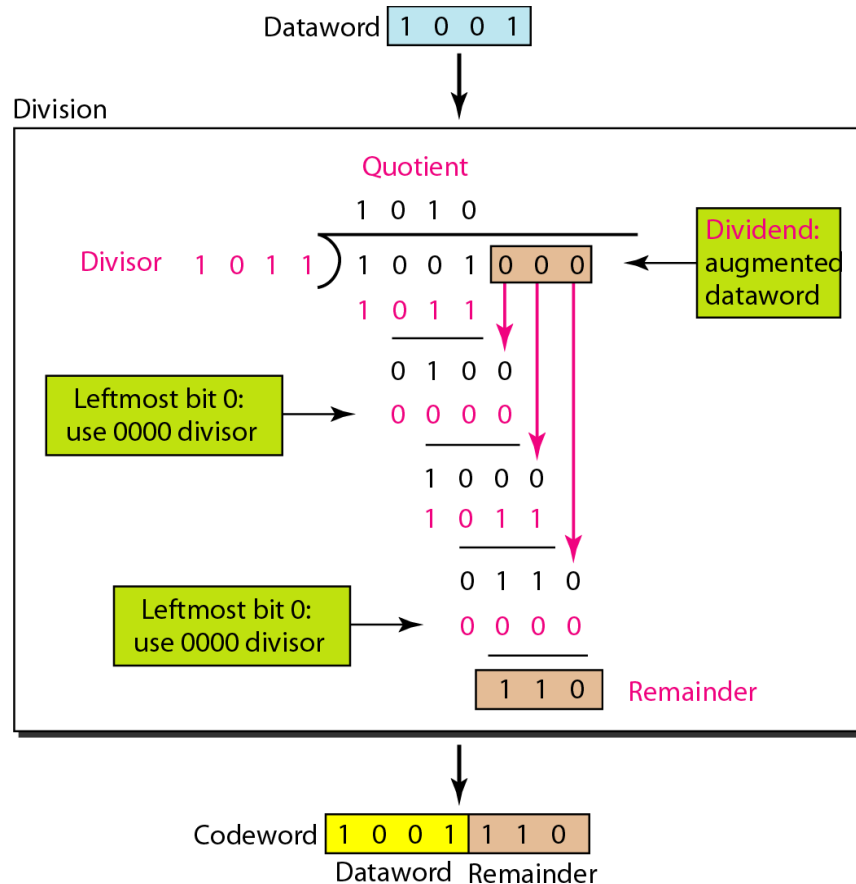
The size of the data word is augmented by adding  $n - k$  (3 here) 0s to the right-hand side of the word. The  $n$ -bit result is fed into the generator. The generator uses a divisor of size  $n - k + 1$  (4 here), predefined and agreed upon. The generator divides the augmented data word by the divisor (modulo-2 division). The quotient of the division is discarded; the remainder ( $r_2r_1r_0$ ) is appended to the data word to create the code word.

The decoder receives the possibly corrupted code word. A copy of all  $n$  bits is fed to the checker which is a replica of the generator. The remainder produced by the checker is a syndrome of  $n - k$  (3 here) bits, which is fed to the decision logic analyzer. The analyzer has a simple function. If the syndrome bits are all as, the 4 leftmost bits of the code word are accepted as the data word (interpreted as no error); otherwise, the 4 bits are discarded (error).

## Encoder

Let us take a closer look at the encoder. The encoder takes the data word and augments it with  $n - k$  number of as. It then divides the augmented data word by the divisor, as shown in Figure.

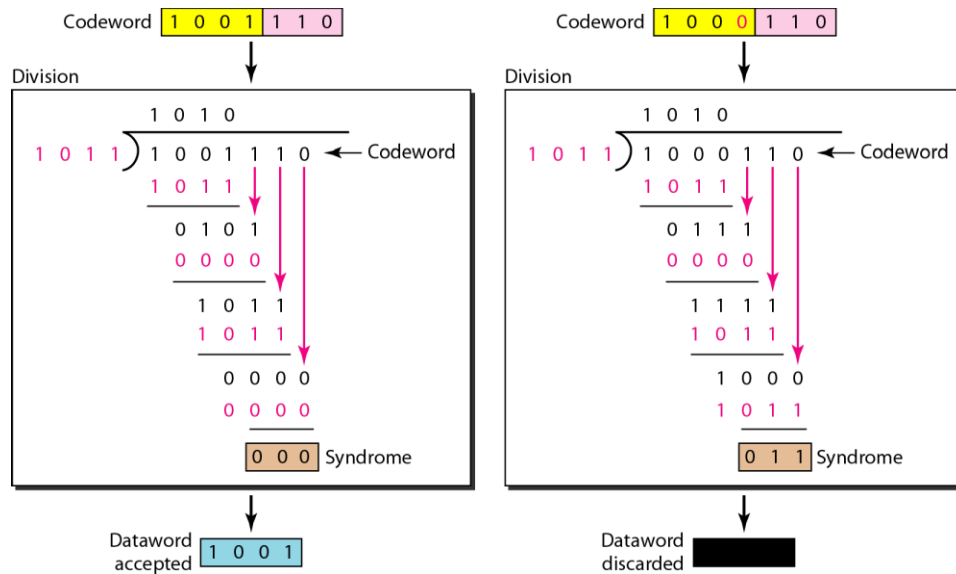
### Division in CRC encoder:



## Decoder

The code word can change during transmission. The decoder does the same division process as the encoder. The remainder of the division is the syndrome. If the syndrome is all 0s, there is no error; the data word is separated from the received code word and accepted. Otherwise, everything is discarded. Figure 10.16 shows two cases: The left hand figure shows the value of syndrome when no error has occurred; the syndrome is 000. The right-hand part of the figure shows the case in which there is one single error. The syndrome is not all 0s (it is 011).

### Division in the CRC decoder for two cases:



### Augmented Dataword

In our paper-and-pencil division process in Figure 10.15, we show the augmented data word as fixed in position with the divisor bits shifting to the right, 1 bit in each step.

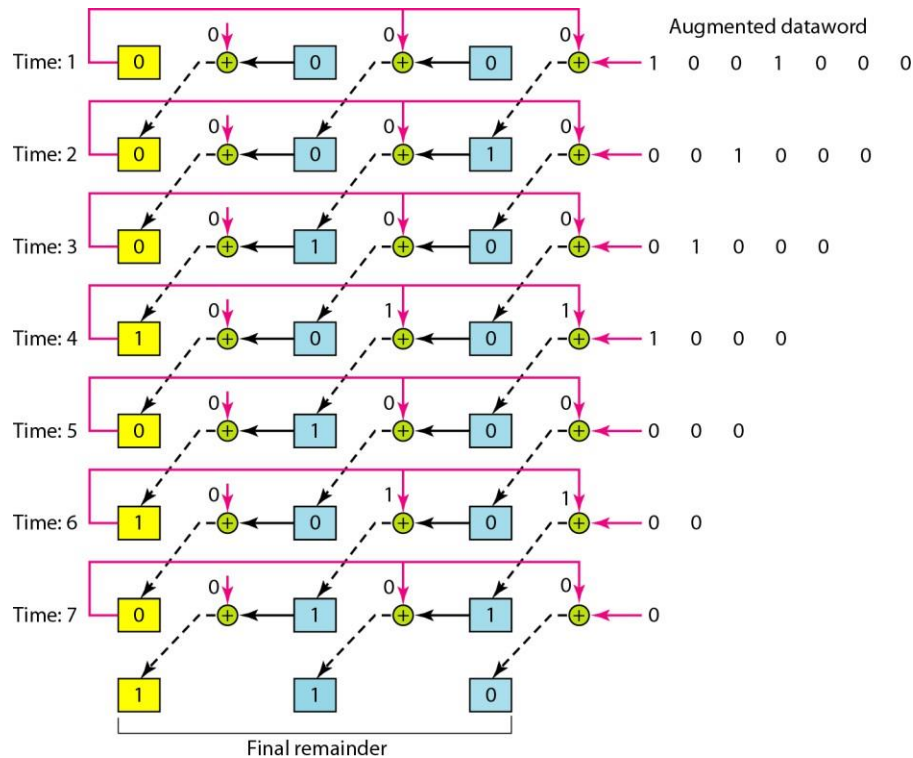
The divisor bits are aligned with the appropriate part of the augmented data word. Now that our divisor is fixed, we need instead to shift the bits of the augmented data word to the left (opposite direction) to align the divisor bits with the appropriate part. There is no need to store the augmented data word bits.

### Remainder

In our previous example, the remainder is 3 bits ( $n - k$  bits in general) in length. We can use three registers (single-bit storage devices) to hold these bits. To find the final remainder of the division, we need to modify our division process. The following is the step-by-step process that can be used to simulate the division process in hardware (or even in software).

1. We assume that the remainder is originally all 0s (000 in our example).
2. At each time click (arrival of 1 bit from an augmented data word), we repeat the following two actions:
  - a. We use the leftmost bit to make a decision about the divisor (011 or 000).
  - b. The other 2 bits of the remainder and the next bit from the augmented data word (total of 3 bits) are XORed with the 3-bit divisor to create the next remainder.

### Simulation of division in CRC encoder:



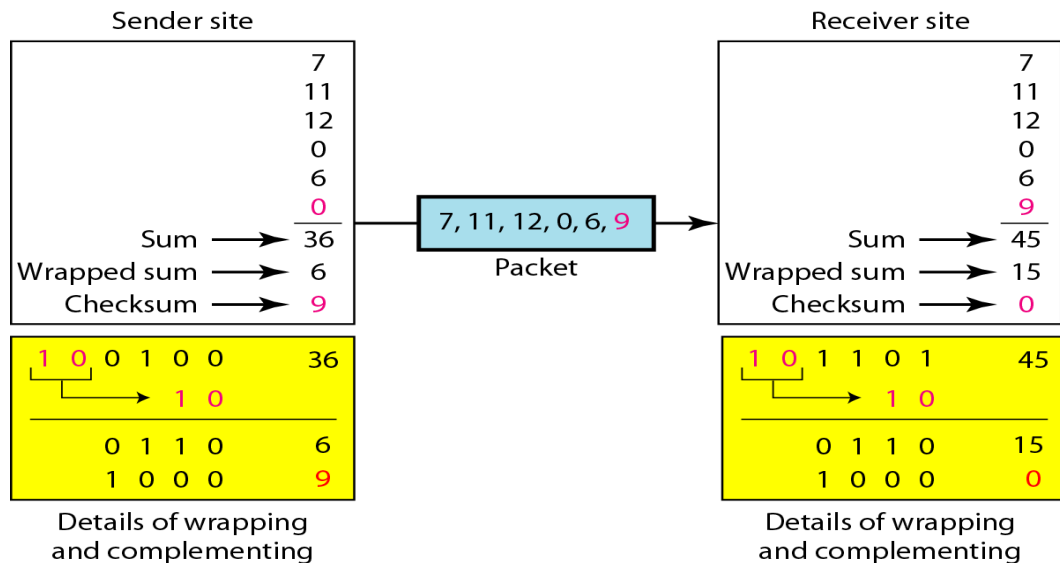
### CHECKSUM:

The checksum is used in the Internet by several protocols although not at the data link layer.

### Implementation of checksum:

The sender initializes the checksum to 0 and adds all data items and the checksum. 36 cannot be expressed in 4 bits. The extra two bits are wrapped and added with the sum to create the wrapped sum value 6.

The sum is then complemented, resulting in the checksum value 9 ( $15 - 6 = 9$ ).



**Example:**

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits. 10101001 00111001

The numbers are added using one's complement

|          |          |
|----------|----------|
| 10101001 | 00111001 |
|          | -----    |
| Sum      | 11100010 |
| Checksum | 00011101 |

The pattern sent is 10101001 00111001 00011101

Now suppose the receiver receives the pattern sent in Example 7 and there is no error. 10101001  
00111001 00011101

When the receiver adds the three sections, it will get all 1s, which, after complementing, is all 0s and shows that there is no error.

|     |          |
|-----|----------|
|     | 10101001 |
|     | 00111001 |
|     | 00011101 |
|     | -----    |
| Sum | 11111111 |

**Complement** 00000000 means that the pattern is OK.

Now suppose there is a burst error of length 5 that affects 4 bits.

10101111 1111001 00011101

When the receiver adds the three sections, it gets

|             |            |
|-------------|------------|
|             | 10101111   |
|             | 11111001   |
|             | 00011101   |
|             | -----      |
| Partial Sum | 1 11000101 |
| Carry       | 1          |
| Sum         | 11000110   |

**Complement** 00111001 the pattern is corrupted.

### Internet Checksum:

#### Sender site:

1. The message is divided into 16-bit words.
2. The value of the checksum word is set to 0.
3. All words including the checksum are added using one's complement addition.
4. The sum is complemented and becomes the checksum.
5. The checksum is sent with the data.

#### Receiver site:

1. The message (including checksum) is divided into 16-bit words.
2. All words are added using one's complement addition.
3. The sum is complemented and becomes the new checksum.
4. If the value of checksum is 0, the message is accepted; otherwise, it is rejected.

#### Example:

|       |   |   |   |  |                    |
|-------|---|---|---|--|--------------------|
| 1     | 0 | 1 | 3 |  | Carries            |
| 4     | 6 | 6 | F |  | (Fo)               |
| 7     | 2 | 6 | 7 |  | (ro)               |
| 7     | 5 | 7 | A |  | (uz)               |
| 6     | 1 | 6 | E |  | (an)               |
| 0     | 0 | 0 | 0 |  | Checksum (initial) |
| <hr/> |   |   |   |  |                    |
| 8     | F | C | 6 |  | Sum (partial)      |
| <hr/> |   |   |   |  |                    |
| 8     | F | C | 7 |  | Sum                |
| 7     | 0 | 3 | 8 |  | Checksum (to send) |

a. Checksum at the sender site

|       |   |   |   |  |                     |
|-------|---|---|---|--|---------------------|
| 1     | 0 | 1 | 3 |  | Carries             |
| 4     | 6 | 6 | F |  | (Fo)                |
| 7     | 2 | 6 | 7 |  | (ro)                |
| 7     | 5 | 7 | A |  | (uz)                |
| 6     | 1 | 6 | E |  | (an)                |
| 7     | 0 | 3 | 8 |  | Checksum (received) |
| <hr/> |   |   |   |  |                     |
| F     | F | F | E |  | Sum (partial)       |
| <hr/> |   |   |   |  |                     |
| 8     | F | C | 7 |  | Sum                 |
| 0     | 0 | 0 | 0 |  | Checksum (new)      |

a. Checksum at the receiver site